Economic-Aware and Sovereignty-Constrained Routing for Enterprise LLM Gateways

Draft v0.4 — two-provider (OpenAI US + Mistral EU) evaluation with a 30-repetition statistical run.

A complet
reproduce

Abstract

Enterprises are adopting large language models (LLMs) faster than they can govern them: they cannot

say which
model,

1. Introduction

LLMs are now embedded in HR assistants, document RAG, developer tooling, and analytical agents.

Each

call has a

1. **Cost opacity.** Teams cannot attribute spend per application/team or decide when a cheaper model

would suff

Existing gateways and FinOps tools mostly *measure and report*. Our thesis is that a control plane at

the **gatew**

Contributions.

- A cons

2. Background and Related Work

References are in `references.bib` (recent, verified); citation keys are shown in brackets.

Model routing and cascades. A growing line of work trades quality for cost by selecting among

models pe
a

LLM serving systems. Orca [yu2022orca], vLLM/PagedAttention [kwon2023pagedattention], SGLang [zheng202

LLM gateways. Envoy AI Gateway [envoyaigateway] and LiteLLM [litellm] provide a unified data path

to multiple

Evaluation. We follow the LLM-as-a-judge methodology of Zheng et al. [zheng2023judge] (MT-Bench /

Chatbot A
we adopt

Regulation. Data-residency and sensitive-data handling are driven by the GDPR [gdpr2016] and the

EU AI Act [

Positioning (Table 0). The following feature matrix makes the delta explicit: prior routing work

optimizes

Capability	**FrugalGPT**	**Hybrid LLM**	**RouteLLM**	**AutoMix**	****Ours****
Cost-aware routing	✓	✓	✓	✓	✓
Quality-aware routing	✓	✓	✓	✓	✓
Learned difficulty predictor	✓	✓	✓	✓	✗ (priors; future: graft)
Hard sovereignty constraints	✗	✗	✗	✗	**✓**
Per-team/namespace budgets	✗	✗	✗	✗	**✓**
Budget-aware graceful degradation	✗	✗	✗	✗	**✓**
Cost attribution (team/ns/app)	✗	✗	✗	✗	**✓**
Managed-vs-self-hosted break-even	✗	✗	✗	✗	**✓**
Gateway/K8s control-plane artifact	✗	✗	✗	✗	**✓**

We do not claim to beat learned routers on the cost–quality frontier; rather, our governance layer is

*orthogon

3. Problem Formulation and Algorithm

A request r carries metadata (team, namespace, sensitivity) and an estimated token footprint. Given

a catalog o

Hard constraint. Reject any m that violates P: its zone is forbidden; or an allow-list is set

and m's

Soft objective. Among C meeting a minimum quality $q_{\min}$ (relaxed only when the budget is

exhaustec

```
\arg\min_{m\in C}\; \alpha\cdot \widehat{c}(m)\,(1+\varepsilon\,\rho) + \beta\,\ell_q(m) + \gamma\,\ell_\lambda(m) + \delta\,\sigma(m)
```

where $\widehat{c}(m)=\text{cost}(m)/\text{cost}(\text{premium})$ is normalized cost, $\rho=used/total$

is budge
expected

Pseudocode and the exact normalization are in `methodology.md`; the implementation is

`experime

4. System Design

Requests flow client → **Envoy AI Gateway** → provider (managed API or self-hosted). The **AI**

Sovereign
budgets,

5. Methodology

Testbed. A Go harness drives the operator's *actual* engines and issues **real** LLM calls across

two prov
gpt-4.1-na

Workloads (W1–W4). HR chatbot (short, sensitive), RAG docs (long context, quality-sensitive), Dev

assistant (

Baselines. B1 premium-static, B2 round-robin, B3 least-cost, B4 static namespace policy, B5 budget

hard-block

Metrics. Cost (total, per request, per token, per team via the operator cost engine, % savings);

quality (LL
the

Reproducibility. Temperature 0; all responses and judge scores cached (`results/cache.json`) so re-runs ar

6. Results

RQ1 — Cost (Figure 2, Table 1)

Relative to

RQ2 — Quality (Figure 3, Table 2)

Within thi

RQ3 — Latency and overhead (Figure 4, Table 3)

The **routi**

RQ4 — Sovereignty (Figure 5, Table 4)

A sovereig

RQ5 — Budget-aware degradation (Figure 6, Table 5)

Under a b
availabili

RQ6 — Managed vs self-hosted break-even (Figure 7) — *modeled*

Using rea
managed

Ablation (Figure 8, Table 7)

Removing
term

Statistical significance (N=30 repetitions)

To move b

Figures `figures/fig_stats_{cost,quality,latency}.png` show means ±95% CI over the 30 repetitions.

7. Discussion

Economic-aware routing is most valuable when a workload mixes easy and hard requests: the router sends

cheap req

8. Threats to Validity

Summarized here; full version in `threats_to_validity.md`. **Internal:** API/latency variability

(mitigated

9. Conclusion and Future Work

Within our preliminary, two-provider, single-pass testbed, a gateway-level, economic-aware,

sovereignt

keeping

Artifact / Reproducibility

Operator + `experimentation/` (datasets, harness, scripts, results, figures, cached responses).

Reproduce

References

See `references.bib` (BibTeX). Key works cited: model routing/cascades — FrugalGPT

[chen2023

Appendix: Figures

Figure 2 — Total cost by routing strategy

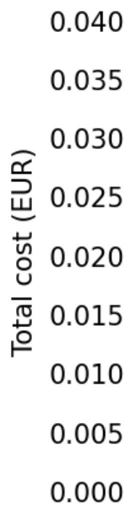

Figure 3 — Quality vs cost

Figure 4 — Latency p95 and mean ±95% CI

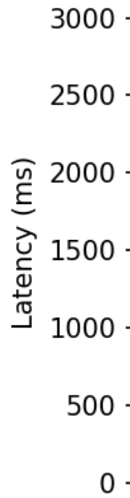

Figure 5 — Sovereignty: cost & violations

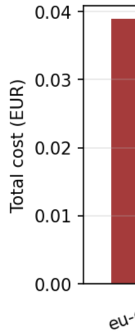

Figure 6 — Budget policy: availability vs overrun

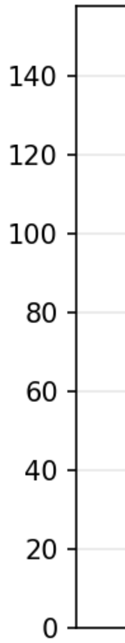

Figure 7 — Managed vs self-hosted break-even (modeled)

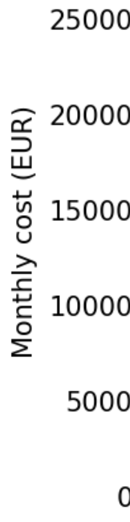

Figure 8 — Ablation of scoring terms

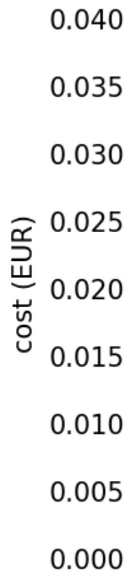

Figure S1 — Cost per rep, mean ±95% CI (N=30)

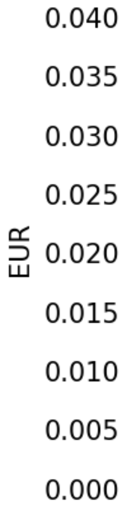

Figure S2 — Quality per rep, mean ±95% CI (N=30)

Figure S3 — Latency per rep, mean ±95% CI (N=30)